Zeluvi – Documento de System Design

Visão geral

O Zeluvi é uma plataforma web e mobile que centraliza comunicação, ocorrências e serviços de um condomínio para três perfis: morador, síndico/administração e portaria. Este documento descreve como o sistema será construído para suportar alto volume e exigências corporativas.

Objetivos técnicos

- Suportar 1 milhão de escritas para cada 2 milhões de leituras (detalhado no documento de Estratégia de Escalabilidade).
- Manter o backend stateless, para escalar com réplicas idênticas.
- Isolar os dados de cada condomínio (multi-tenant) e proteger dados pessoais conforme a LGPD.
- Oferecer interface acessível, inclusive para idosos, seguindo o guia de estilo do projeto.

Dentro do escopo: módulos de Comunicação, Ocorrências e Serviços, os três perfis de usuário e os diferenciais com IA (Copiloto do Síndico, Motor de Automações, Painel Executivo, Morador 360° e Assembleias Digitais com transcrição por IA).

Fora do escopo: fabricação de hardware (o sistema só se integra por API a equipamentos existentes), contabilidade bancária profunda equivalente a um ERP e portaria remota sem supervisão humana.

Requisitos

A arquitetura é guiada por seis requisitos não funcionais, listados abaixo com metas propostas que a equipe deve validar.

Requisito	Meta proposta	Como atendemos
Disponibilidade	99,9% ao mês	Réplicas da aplicação em duas zonas de disponibilidade e banco com failover automático
Latência	p95 abaixo de 300 ms na leitura e de 500 ms na escrita	Cache em Redis, índices no banco e trabalho pesado em fila
Escalabilidade	1 milhão de escritas para cada 2 milhões de leituras	Escala horizontal, réplicas de leitura e filas (ver Estratégia de Escalabilidade)

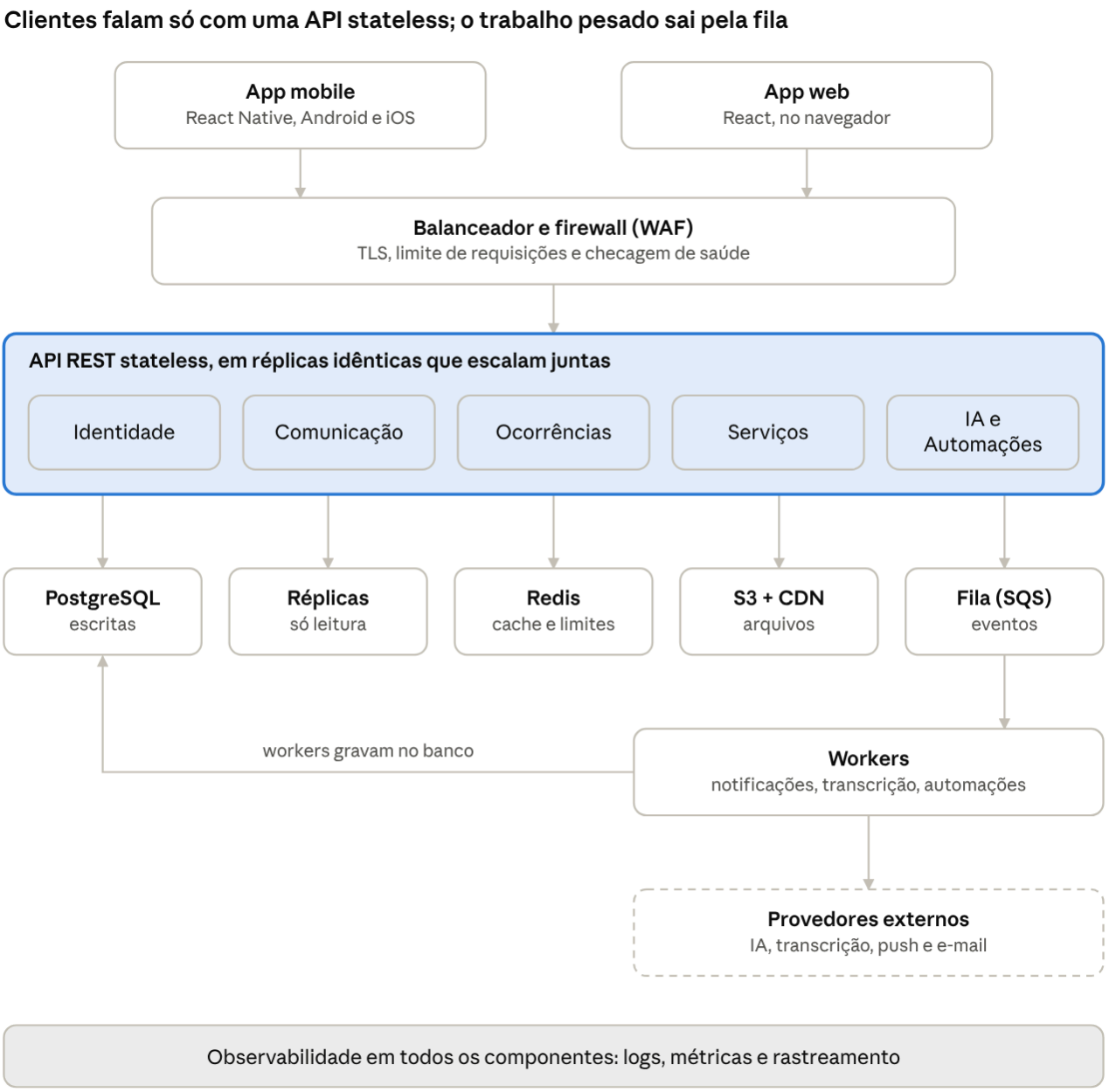
Requisito	Meta proposta	Como atendemos
Segurança e LGPD	TLS 1.2 ou superior, criptografia em repouso e trilha de auditoria	Autenticação por token, controle de acesso por perfil e isolamento por condomínio
Recuperação	Perda máxima de 15 min e retorno em até 1 h	Backups contínuos com restauração para qualquer ponto no tempo
Acessibilidade	WCAG 2.2 nível AA	Componentes do guia de estilo, com textos grandes e contraste alto

Requisitos funcionais por perfil

- **Morador:** relatar ocorrências, receber comunicados, fazer reservas, avisar mudanças e acompanhar solicitações.
- **Síndico / Administração:** gerenciar ocorrências e reservas, enviar comunicados, administrar cadastros de moradores e usar os recursos de IA (Copiloto, Automações, Painel Executivo, Morador 360° e Assembleias Digitais).
- **Portaria:** registrar recebimento e entrega de encomendas, registrar avisos de mudança, acompanhar ocorrências relevantes e apoiar a comunicação do condomínio.

Arquitetura de alto nível

A arquitetura separa o que o usuário espera ver na hora (a API) do que pode esperar alguns segundos (os workers, alimentados por uma fila).



arquitetura de alto nível · clientes, API, dados, fila e workers

Os apps chegam à API por um balanceador. A API grava no PostgreSQL, lê do cache e das réplicas e publica eventos na fila. Os workers cuidam de notificações, transcrições e automações e chamam provedores externos quando precisam.

Componentes e módulos

O backend é dividido em cinco módulos que compartilham o mesmo banco, mas só conversam entre si por interfaces internas e por eventos. Quando algo relevante acontece (uma ocorrência criada, uma reserva aprovada), o módulo publica um evento numa fila, e os módulos de Comunicação e de Automações reagem a ele sem travar a requisição de quem fez a ação.

Módulo	O que faz	Quem usa
Identidade e Acesso	Login, perfis e cadastro de moradores e unidades	Todos; o síndico administra os cadastros
Comunicação	Comunicados, notificações e mensagens diretas entre moradores e administração	Síndico envia; morador e portaria recebem
Ocorrências	Relato e acompanhamento de problemas: manutenção, limpeza, elevador, iluminação e outros	Morador abre; síndico gerencia; portaria acompanha as relevantes
Serviços	Reserva do salão e de áreas comuns, agendamento do elevador de serviço, encomendas e avisos de mudança	Morador reserva; síndico aprova; portaria registra encomendas e mudanças
IA e Automações	Copiloto do Síndico, Motor de Automações, Painel Executivo, Morador 360° e Assembleias Digitais com transcrição	Síndico e administração

Modelo de dados

O banco é relacional (PostgreSQL) e gira em torno do condomínio: toda tabela tem um identificador do condomínio, um identificador único universal (UUID) e as datas de criação e alteração. A segurança em nível de linha do PostgreSQL impede que uma consulta enxergue dados de outro condomínio, mesmo que o código erre o filtro.

Entidade	Campos principais	Observação
Condomínio	nome, endereço, configurações	Raiz de todos os dados
Unidade	bloco, número, condomínio	Liga moradores a um endereço
Usuário	nome, e-mail, perfil, unidade	Perfil: morador, síndico ou portaria
Comunicado e Leitura	título, texto, público, quem leu e quando	A confirmação de leitura gera uma escrita por morador
Ocorrência e Histórico	tipo, descrição, local, status, responsável	Cada mudança de status vira uma linha de histórico

Entidade	Campos principais	Observação
Área comum e Reserva	área, data, horário, status, regras de uso	Restrição no banco impede reservas sobrepostas
Encomenda	unidade, recebida em, entregue em, quem retirou	Registrada pela portaria
Aviso de mudança	unidade, data, tipo (entrada ou saída)	Registrado pelo morador e visto pela portaria
Mensagem	conversa, remetente, texto	Canal direto entre morador e administração
Assembleia e Transcrição	pauta, data, texto transcrito, ata	A transcrição por IA é gerada por um worker
Regra de automação	gatilho, condição, ação, execuções	Usada pelo Motor de Automações
Evento de auditoria	quem, o quê, quando, de onde	Somente inserção; nunca é alterado

Arquivos (fotos de ocorrências, anexos de comunicados) ficam no armazenamento de objetos, e o banco guarda só o endereço e os metadados.

Segurança, perfis e permissões

A segurança se apoia em três barreiras: quem é o usuário, o que o perfil dele pode fazer e de qual condomínio ele pode ver dados.

- **Perfis:** morador, síndico/administração e portaria. As permissões de cada perfil seguem a matriz de acesso do mapa de telas e são conferidas em toda rota da API, não só escondidas na tela.
- **Autenticação:** senha com hash Argon2, token de acesso de curta duração e refresh token rotativo. Proposta: verificação em duas etapas obrigatória para síndico e administração, que podem aprovar, apagar e enviar comunicados.
- **Isolamento entre condomínios:** filtro no código e segurança em nível de linha no banco, como segunda trava.
- **Proteção dos dados:** TLS 1.2 ou superior em todo tráfego, criptografia em repouso no banco e nos arquivos, e segredos guardados em um cofre (AWS Secrets Manager), nunca no código.

- **LGPD:** coletar só o necessário, registrar o consentimento, permitir que o titular consulte, corrija e apague seus dados e registrar cada acesso a dados pessoais. Assembleias transcritas por IA avisam os participantes antes de gravar.
- **Auditoria:** ações sensíveis (aprovar reserva, encerrar ocorrência, alterar cadastro) geram um evento que só admite inserção.
- **Proteção de borda:** firewall de aplicação (WAF), limite de requisições, validação de toda entrada e verificação automática de dependências vulneráveis a cada entrega.
- **Hardware existente:** câmeras e catracas, quando integradas, usam chaves de API próprias e com permissão mínima.

Infraestrutura, implantação e observabilidade

O sistema roda na AWS, na região de São Paulo, o que mantém os dados no Brasil (importante para a LGPD) e reduz a latência para os moradores.

Infraestrutura

- Rede privada (VPC): só o balanceador fica em sub-rede pública; aplicação, banco e cache ficam em sub-redes privadas.
- Aplicação em contêineres (ECS Fargate), com réplicas em duas zonas de disponibilidade.
- Toda a infraestrutura descrita como código (Terraform), para recriar qualquer ambiente do zero.

Implantação

1. A cada alteração, o GitHub Actions roda os testes e o build.
2. A versão vai para homologação e passa por testes automáticos.
3. Após aprovação, a produção recebe a versão de forma gradual, trocando as réplicas aos poucos. Se os alertas dispararem, a versão anterior volta automaticamente.

Observabilidade

- Logs estruturados, com um identificador que acompanha a requisição do app até o banco.
- Métricas principais: latência p95, taxa de erro, tamanho da fila, CPU e conexões do banco.
- Rastreamento distribuído (OpenTelemetry) e alertas para a equipe quando uma meta de latência ou de disponibilidade é violada.

Riscos e decisões em aberto

O maior risco é dimensionar errado por não saber em que período a meta de 1 milhão de escritas e 2 milhões de leituras se aplica. Por isso o sistema é projetado para o cenário mais pesado (por hora). Os demais riscos estão abaixo.

Risco	Impacto	Resposta
Custo das chamadas de IA	Conta mensal alta por condomínio	Limite de uso por condomínio, processamento assíncrono e cache de respostas repetidas
Provedor de IA fora do ar	Copiloto e transcrição indisponíveis	Falha isolada: comunicados, ocorrências e reservas funcionam sem a IA
Gravação e transcrição de assembleias (LGPD)	Risco jurídico	Aviso e consentimento antes de gravar, e prazo de retenção definido
Sinal fraco no celular	Morador perde uma ação	Cache de leitura e repetição segura de escritas com chave de idempotência
Equipe pequena para operar o sistema	Falhas sem resposta rápida	Serviços gerenciados, infraestrutura como código e alertas

Decisões em aberto

- Confirmar o período da meta de carga (por dia ou por hora).
- Validar a nuvem (AWS foi a proposta) e escolher os provedores de IA e de notificação push.
- Definir o prazo de retenção de dados pessoais, transcrições e auditoria.

Próximo documento: o Mapeamento de Integrações (API de pagamento e comunicação por WebSocket) complementa este desenho.